SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA0624	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	LOKESH S	**Reg. No.:**	192572151
Section / Batch:	CSE(AI) / 2ND YEAR	**Date:**	08/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Case Study					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

DESIGN AND ANALYSIS OF ALGORITHMS – CASE STUDY ANSWERS

1. Case Study: E-Commerce Product Sorting System – Merge Sort

INTRODUCTION

An e-commerce platform may contain millions of products that must be sorted according to attributes such as price, customer rating, popularity, or product name. Efficient sorting is essential because users expect search and filtering results to be displayed quickly. Merge Sort is a suitable algorithm for handling large datasets because it follows the Divide and Conquer strategy and guarantees O(n log n) time complexity in all major cases.

KEY ISSUES

The major issues in an e-commerce sorting system are:

1. Large data volume: Millions of products must be processed.
2. Scalability: The algorithm should continue to perform efficiently as the dataset grows.
3. Stability: Products having equal prices or ratings should maintain their original relative order.
4. Performance: Sorting should not become extremely slow for already sorted or reverse-sorted data.
5. External data: Very large datasets may not fit completely into main memory.

DIVIDE AND CONQUER APPROACH

Merge Sort divides the product dataset into smaller parts until each part contains a single product. The process consists of three stages:

1. Divide:

The array is divided into two approximately equal halves.

2. Conquer:

Each half is recursively sorted using Merge Sort.

3. Combine:

The two sorted halves are merged into one sorted list.

For example:

[Product1, Product2, Product3, Product4]
is divided into:
[Product1, Product2] and [Product3, Product4]
These are further divided and sorted. Finally, the sorted portions are merged.

ALGORITHM

1. Divide the product list into two halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Compare products from both sorted halves.
5. Place the smaller/higher-priority product into the result.
6. Continue until all products are merged.

Time Complexity

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

The recurrence is:
$T(n) = 2T(n/2) + O(n)$
Therefore:
$T(n) = O(n \log n)$

Space Complexity
Standard Merge Sort requires:
O(n) additional memory.

Stability
Merge Sort is a stable sorting algorithm when implemented correctly. If two products have the same price, their previous ordering can be preserved.
For example:
Laptop A – ₹50,000 – Rating 4.5
Laptop B – ₹50,000 – Rating 4.2
After sorting by price, A can remain before B.
Feasible Solution
The e-commerce platform can use external Merge Sort when the dataset is too large for RAM. Product records can be divided into manageable chunks, sorted independently, stored temporarily, and then merged.
For multiple sorting attributes, the system can use a stable comparator such as:
Price → Rating → Product ID

Advantages

- Guaranteed O(n log n) performance.
- Stable sorting.
- Suitable for very large datasets.
- Works efficiently with external storage.
- Performance is predictable.
- Easy to parallelize because different portions can be sorted independently.

CONCLUSION
Merge Sort is highly suitable for an e-commerce product sorting system because it provides predictable O(n log n) performance and stability. Its additional memory requirement is a disadvantage for in-memory sorting, but external Merge Sort can solve the problem for millions of records. Therefore, it is an effective and scalable choice for large-scale product sorting.

2. Case Study: Real-Time Ranking System – Quick Sort

Introduction

A real-time ranking system such as a leaderboard continuously sorts users according to scores. The system requires fast sorting because rankings may change frequently. Quick Sort is a suitable algorithm because it is generally fast in practice and can perform in-place sorting with low additional memory.

Key Issues

The main issues are:

1. Frequent score updates.
2. Fast response requirements.
3. Pivot selection.
4. Worst-case performance.
5. Large number of users.
6. Repeated sorting operations.

Divide and Conquer Approach

Quick Sort uses three major steps:

1. Select a pivot:

Choose an element from the array.

2. Partition:

Place elements greater than or smaller than the pivot on appropriate sides.

3. Recursively sort:

Apply Quick Sort to the left and right partitions.

For a leaderboard sorted by descending score:

[50, 90, 70, 40, 100]

If 70 is selected as the pivot:

[50, 40] 70 [90, 100]

The two partitions are recursively sorted.

Pivot Selection

Pivot selection strongly affects Quick Sort performance.

Possible strategies include:

- First element.
- Last element.
- Middle element.
- Random element.
- Median-of-three.

A poor pivot can produce highly unbalanced partitions.

Complexity

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$

The best case occurs when the pivot divides the array into approximately equal portions.
The worst case occurs when the pivot repeatedly becomes the smallest or largest element.
Worst-Case Example
For an already sorted list:
10, 20, 30, 40, 50
If the first element is always selected as pivot, partitions become:
0 elements | Pivot | n-1 elements
This produces:
$O(n^2)$
performance.

Optimized Solution
Use randomized Quick Sort.
Instead of always selecting the first or last element, randomly select the pivot.
This reduces the probability of repeatedly obtaining extremely unbalanced partitions.
Another improvement is median-of-three pivot selection, where the median of the first, middle, and last elements is selected.
For very small partitions, Insertion Sort can also be used to reduce recursion overhead.

Advantages

- Average O(n log n).
- Usually faster than Merge Sort for in-memory data.
- Requires only O(log n) average auxiliary stack space.
- Good cache performance.
- Suitable for ranking systems requiring fast sorting.
-

Conclusion
Quick Sort is appropriate for a real-time ranking system because of its excellent average-case performance and low memory usage. However, naive pivot selection can cause $O(n^2)$ performance. Therefore, randomized pivot selection, three-way partitioning, and hybrid techniques should be used to make the system more robust.

3. Case Study: Search Engine Query Processing – Binary Search

Introduction

A search engine may maintain a sorted index containing millions of keywords. When a user enters a query, the system must quickly determine whether the keyword exists and retrieve its associated information. Binary Search is highly efficient for searching sorted data because it repeatedly eliminates half of the remaining search space.

Key Requirements

The system requires:

1. Sorted keyword data.
2. Fast retrieval.
3. Large-scale data handling.
4. Low search latency.
5. Efficient memory usage.

Divide and Conquer Approach

Binary Search follows the Divide and Conquer principle.

The algorithm:

1. Finds the middle element.
2. Compares it with the target keyword.
3. If equal, the search is successful.
4. If the target is smaller, search the left half.
5. If the target is larger, search the right half.
6. Repeat until the element is found or the search range becomes empty.

Example

Consider:

[Apple, Banana, Computer, Database, Engine, Network, Python]
Search for Engine.
Middle element:
Database
Since Engine comes after Database, search the right half:
[Engine, Network, Python]
Middle element:
Network
Engine comes before Network.
Search:
[Engine]
The keyword is found.

Time Complexity

Case	Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

The search space is reduced by half at every step.
For example, with approximately one million records:
1,000,000 → 500,000 → 250,000 → ...
Only around 20 comparisons are required in the worst case because:
$\log_2(1{,}000{,}000) \approx 20$

Space Complexity
Iterative Binary Search requires:
O(1) auxiliary space.
Recursive Binary Search requires:
O(log n) stack space.

System Design
The search engine can maintain:
Keyword → Document IDs/URLs
in a sorted index.

When a query arrives:
User Query → Normalize → Binary Search Index → Retrieve Matching Entry → Return Results
For extremely large search engines, specialized indexes and hash-based structures may also be used, but Binary Search is highly suitable when the requirement is specifically searching a sorted array or indexed list.

Advantages

- Very fast searching.
- O(log n) time.
- Simple implementation.
- Low memory requirement.
- Efficient for millions of sorted records.

Limitation
The primary requirement is that the data must be sorted. If the dataset changes frequently, maintaining sorted order may require additional processing.

Conclusion
Binary Search is an optimal basic searching technique for a static sorted keyword index because it reduces the search space by half at each step. Its O(log n) worst-case complexity makes it significantly faster than linear search for large sorted datasets.

4. Case Study: Image Processing System – Matrix Multiplication

Introduction

Image processing applications use matrices to represent pixels, transformations, filters, and other operations. Large images and complex transformations can require multiplication of large matrices. Traditional matrix multiplication has a time complexity of $O(n^3)$, which becomes expensive as matrix size increases.

Divide and Conquer techniques such as Strassen's Matrix Multiplication can reduce the number of multiplication operations.

Computational Challenges

The major challenges are:

1. Large matrix dimensions.
2. High number of multiplication operations.
3. High execution time.
4. Memory usage.
5. Repeated transformations.
6. Processing high-resolution images.

Traditional Matrix Multiplication

For two n × n matrices:

$C[i][j] = \Sigma A[i][k] \times B[k][j]$

Three nested loops are generally used.

Time complexity:

$O(n^3)$

Divide and Conquer Matrix Multiplication

Divide each matrix into four submatrices:

A =

A11 A12

A21 A22

and B =

B11 B12

B21 B22

The resulting matrix is:

C =

C11 C12

C21 C22

Each submatrix is recursively processed.

Strassen's Method
Traditional divide-and-conquer matrix multiplication requires 8 recursive multiplications. Strassen's method reduces this to 7 multiplications by using additional additions and subtractions. Its recurrence is approximately:
$T(n) = 7T(n/2) + O(n^2)$
Therefore:
$T(n) = O(n^{\log_2 7})$
Since:
$\log_2 7 \approx 2.807$
the complexity is:
$O(n^{2.807})$

Comparison

Method	Time Complexity
Traditional	$O(n^3)$
Strassen	$O(n^{2.807})$

Suitable Solution
For large matrices, the image processing system can use Strassen's method when matrix dimensions are sufficiently large. For smaller matrices, conventional multiplication can be used because Strassen's additional operations and recursion can create overhead.
A hybrid implementation is therefore preferable:
Large Matrix → Strassen
Small Matrix → Traditional Multiplication

Advantages
- Fewer multiplication operations.
- Better theoretical performance for large matrices.
- Suitable for computationally intensive applications.
- Divide-and-conquer structure allows parallel implementation.

Limitations
- More additions and subtractions.
- More complex implementation.
- Additional memory requirements.
- Not always faster for small matrices.
- Numerical considerations may arise with floating-point data.

Conclusion
Strassen's algorithm can improve the performance of large matrix operations by reducing the multiplication count from eight to seven at each recursive level. Its $O(n^{2.807})$ complexity is better than traditional $O(n^3)$. A hybrid approach provides the best practical solution.

5. Case Study: Data Analytics Platform – Merge Sort vs Quick Sort

Introduction
A data analytics platform processes large datasets for reports, dashboards, and statistical analysis. Sorting may be required according to sales, revenue, dates, customer IDs, or other attributes. Merge Sort and Quick Sort are both Divide and Conquer algorithms, but they have different characteristics regarding speed, memory usage, stability, and worst-case performance.

Merge Sort
Merge Sort divides the dataset into two halves, recursively sorts them, and merges the sorted halves.

Time complexity:
- Best: O(n log n)
- Average: O(n log n)
- Worst: O(n log n)

Space:
O(n)
It is stable.

Quick Sort
Quick Sort selects a pivot, partitions the dataset, and recursively sorts the resulting partitions.

Time complexity:
- Best: O(n log n)
- Average: O(n log n)
- Worst: $O(n^2)$

Average auxiliary space is approximately:
O(log n)
It is generally not stable.

Comparison

Feature	Merge Sort	Quick Sort
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	O(n log n)	$O(n^2)$
Extra Memory	O(n)	O(log n) average
Stable	Yes	Usually No
In-place	No	Generally Yes
Large External Data	Excellent	Less suitable
Practical Speed	Good	Usually very good

Performance Conditions

Merge Sort is preferred when:

- Stability is required.
- Data is very large.
- Predictable performance is important.
- External sorting is required.

Quick Sort is preferred when:

- Data is primarily in memory.
- Memory usage must be minimized.
- Fast average performance is required.
- Stability is not important.

Proposed Solution

For a large analytics platform, Merge Sort should be selected when reports require stable ordering and datasets may exceed memory.
For smaller in-memory datasets where speed and memory are more important, optimized Quick Sort can be used.
A practical system can use a hybrid sorting strategy based on dataset size and requirements.

Conclusion

Neither algorithm is universally better. Merge Sort provides guaranteed O(n log n) performance and stability, while Quick Sort generally provides excellent practical speed with lower memory usage. For a large reporting platform where data stability and predictable performance are important, Merge Sort is the safer primary choice.